Data Structures and American Option Pricing

FRE6883 – Recitation 8

Store the tree, compare the choices, locate early exercise

From European backward induction to optimal stopping

European value

discounted continuation only

$\downarrow$

American value

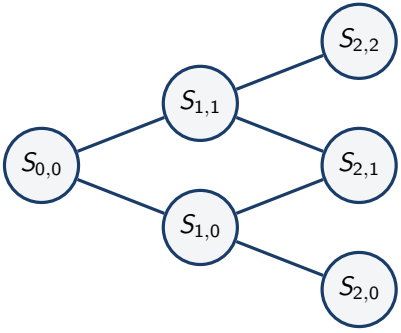
$\max\{\text{continue}, \text{exercise}\}$

By the end

You can implement the Snell recursion with `std::vector`, build a triangular price tree, record an exercise policy, and choose a suitable STL container for a simple task.

We keep the option hierarchy from Recitation 7 and change the pricing algorithm and data structures.

The holder chooses at every node



At each nonterminal node:
compare exercise now
with discounted continuation.

American right: the holder may exercise at any node up to and including expiry.

The Snell recursion

At expiry

$$H(N,i) = h\big(S(N,i)\big), \qquad i = 0,\ldots,N.$$

At an earlier node

$$H(n,i) = \max\left\{\frac{qH(n+1,i+1) + (1-q)H(n+1,i)}{R}, h\big(S(n,i)\big)\right\}.$$

The two quantities are the continuation value and the immediate exercise value.

GUIDED CODE One Snell node

Compute before voting

At node (n, i), suppose

$$q = 0.55, \quad R = 1.02, \quad H(n+1, i) = 18, \quad H(n+1, i+1) = 6.$$

The immediate exercise value is 12.

Questions

What is the continuation value? What is $H(n, i)$? Does the holder exercise?

$$\text{ContVal} = \frac{0.55(6) + 0.45(18)}{1.02} = 11.176\ldots$$

$$H(n, i) = \max\{11.176\ldots, 12\} = 12.$$

Decision

Immediate exercise is optimal because its payoff exceeds the value of keeping the option alive.

GUIDED CODE Read the update

Classify A and B

```
// A
Price[i] = (q * Price[i + 1]
          + (1.0 - q) * Price[i]) / R;

// B
double cont = (q * Price[i + 1]
             + (1.0 - q) * Price[i]) / R;
double exercise = Payoff(S);
Price[i] = (cont > exercise) ? cont : exercise;
```

Question

Which update prices a European option? Which implements the Snell envelope?

02 | European or American recurrence? – Solution

A: European

Only continuation is allowed before expiry.

B: American

The holder receives the better of continuation and exercise.

Useful bound

With the same payoff and model, the American value is at least the corresponding European value because it offers every European strategy plus early exercise.

03 | Predict the four lecture prices – Problem

GUIDED CODE Financial interpretation

Call or put: where is the premium?

```
calc.UpdateOption(&call);
double euroCall = calc.PriceByCRR(model);
double amerCall = calc.PriceBySnell(model);

calc.UpdateOption(&put);
double euroPut = calc.PriceByCRR(model);
double amerPut = calc.PriceBySnell(model);
```

Vote

Which American price equals its European counterpart for the lecture inputs? Which one is strictly larger?

03 | Predict the four lecture prices – Solution

Contract	European	American
Call	21.68	21.68
Put	11.43	11.72

Interpretation for this model

The put has a positive early-exercise premium. The non-dividend-paying call does not benefit from early exercise here.

04 | Complete the Snell update – Problem

GUIDED CODE Backward induction

Fill the exercise comparison

```
double contVal =
    (q * Price[i + 1] + (1.0 - q) * Price[i])
    / Model.GetR();

double exerciseVal = pOption->Payoff(
    Model.CalculateAssetPrice(n, i));

Price[i] = /* American value at node (n,i) */;
```

04 | Complete the Snell update – Solution

```
Price[i] = exerciseVal;
if (contVal > Price[i])
    Price[i] = contVal;
```

Equivalent mathematical operation

$$\texttt{Price[i]} = \max\{\texttt{contVal}, \texttt{exerciseVal}\}.$$

The course implementation uses an `if` statement and requires no change to the polymorphic payoff interface.

Why replace the raw array with `std::vector`?

```cpp
double* Price = new double[N + 1];

// ... pricing ...

delete[] Price;
```

```cpp
#include <vector>

std::vector<double> Price(N + 1);

// automatic cleanup
```

Same indexing, managed lifetime

A vector owns its elements, knows its size, and releases its storage automatically when it leaves scope.

GUIDED CODE Construction and push back — Track size and contents

```
std::vector<double> prices(3);
prices[0] = 10.0;
prices[1] = 20.0;
prices.push_back(40.0);

std::cout << prices.size() << std::endl;
std::cout << prices[2] << " " << prices[3] << std::endl;
```

Question

What is printed? What value did the constructor place in `prices[2]`?

05 | Predict the vector state – Solution

```
// After construction: {0.0, 0.0, 0.0}
// After assignments:  {10.0, 20.0, 0.0}
// After push_back:    {10.0, 20.0, 0.0, 40.0}

// Output:
4
0 40
```

Two different operations

`vector<double> prices(3)` creates three elements. `push_back` appends a new fourth element.

06 | Find the vector indexing bug – Problem

GUIDED CODE Bounds

Identify the first invalid access

```
std::vector<double> Price(N + 1);

for (int i = 0; i <= N + 1; ++i)
{
    Price[i] = 0.0;
}
```

Questions

What are the valid indices? Which loop condition is correct? Does `operator[]` automatically check the bound?

06 | Find the vector indexing bug – Solution

```
std::vector<double> Price(N + 1);

for (int i = 0; i <= N; ++i)
{
    Price[i] = 0.0;
}
```

Valid range

A vector of size $N + 1$ has indices $0, 1, \ldots, N$. `operator[]` does not perform automatic bounds checking.

`Price.at(i)` performs a checked access and throws `std::out_of_range` when the index is invalid.

07 | Sized vector or `push_back`? – Problem

GUIDED CODE Construction choice

Which loop is correct?

```cpp
// A
std::vector<double> a(N + 1);
for (int i = 0; i <= N; ++i)
    a.push_back(payoff(i));

// B
std::vector<double> b(N + 1);
for (int i = 0; i <= N; ++i)
    b[i] = payoff(i);

// C
std::vector<double> c;
for (int i = 0; i <= N; ++i)
    c.push_back(payoff(i));
```

07 | Sized vector or `push_back`? – Solution

A: wrong size

Starts with $N + 1$ elements, then appends $N + 1$ more.

B: valid

Create the final size, then assign by index.

C: valid

Start empty and append each element.

The lecture pricer uses B because the required size $N + 1$ is already known.

08 | Initialize terminal payoffs – Problem

GUIDED CODE Vector in the CRR engine

Complete the loop

```
int N = pOption->GetN();
std::vector<double> Price(/* correct size */);

for (int i = 0; i <= N; ++i)
{
    double z = /* asset value at node (N,i) */;
    Price[i] = /* polymorphic payoff */;
}
```

08 | Initialize terminal payoffs – Solution

```
int N = pOption->GetN();
std::vector<double> Price(N + 1);

for (int i = 0; i <= N; ++i)
{
    double z = Model.CalculateAssetPrice(N, i);
    Price[i] = pOption->Payoff(z);
}
```

Recitation 7 still matters

The vector stores values. Virtual dispatch still selects the call or put payoff through `pOption`.

A vector of vectors can store the whole tree

```
std::vector< std::vector<double> > priceTree;

priceTree.resize(N + 1);
for (int n = 0; n <= N; ++n)
    priceTree[n].resize(n + 1);
```

Row `n` stores time step n. Its valid node indices are

$$i = 0, \ldots, n.$$

Triangular shape

Row sizes are $1, 2, 3, \ldots, N + 1$.

09 | Complete the triangular allocation – Problem

GUIDED CODE Two-dimensional vector

Fill both row sizes

```
std::vector< std::vector<double> > priceTree;
std::vector< std::vector<int> > stoppingTree;

priceTree.resize(/* number of time steps */);
stoppingTree.resize(/* number of time steps */);

for (int n = 0; n <= N; ++n)
{
    priceTree[n].resize(/* nodes in row n */);
    stoppingTree[n].resize(/* nodes in row n */);
}
```

09 | Complete the triangular allocation – Solution

```
priceTree.resize(N + 1);
stoppingTree.resize(N + 1);

for (int n = 0; n <= N; ++n)
{
    priceTree[n].resize(n + 1);
    stoppingTree[n].resize(n + 1);
}
```

Meaning of the indices

`priceTree[n][i]` stores $H(n, i)$. `stoppingTree[n][i]` records the exercise decision at the same node.

10 | Identify the two child nodes – Problem

GUIDED CODE Tree indexing — Complete the continuation value

```
double contVal =
    (q * priceTree[/* up child */][/* up index */]
     + (1.0 - q)
       * priceTree[/* down child */][/* down index */])
    / Model.GetR();
```

Question

From node (n, i), which two entries in row $n + 1$ are required?

10 | Identify the two child nodes – Solution

```
double contVal =
    (q * priceTree[n + 1][i + 1]
     + (1.0 - q) * priceTree[n + 1][i])
    / Model.GetR();
```

$$(n, i) \longrightarrow (n+1, i) \text{ down}, \qquad (n, i) \longrightarrow (n+1, i+1) \text{ up}.$$

Index check

When $i \leq n$, both i and $i+1$ are valid in the next row, whose size is $n+2$.

The option hierarchy is unchanged

```
class Option {
protected:
    int N;
public:
    Option(int N_) : N(N_) {}
    int GetN() const { return N; }
    virtual double Payoff(double z) const = 0;
    virtual ~Option() = 0;
};

class Call : public Option { /* K and call payoff */ };
class Put  : public Option { /* K and put payoff  */ };
```

New algorithm, same contract

Both `PriceByCRR` and `PriceBySnell` ask the selected option for `GetN()` and `Payoff(z)`.

11 | Add the American pricing method – Problem

GUIDED CODE Class interface — Complete the declaration

```
class OptionCalculation {
private:
    Option* pOption;
public:
    double PriceByCRR(
        const BinomialTreeModel& Model);

    /* declare PriceBySnell with the same model input */
};
```

Question

Why does American versus European belong in the pricing method rather than in the `Put` class?

11 | Add the American pricing method – Solution

```
class OptionCalculation {
private:
    Option* pOption;
public:
    double PriceByCRR(
        const BinomialTreeModel& Model);

    double PriceBySnell(
        const BinomialTreeModel& Model);
};
```

Separation of concerns

`Put::Payoff` defines the contract payoff. The selected calculation method determines whether early exercise is permitted.

12 | Complete the terminal row – Problem

GUIDED CODE PriceBySnell

Initialize at expiry

```
double q = Model.RiskNeutProb();
int N = pOption->GetN();
std::vector<double> Price(N + 1);

for (int i = 0; i <= N; ++i)
{
    Price[i] = /* H(N,i) */;
}
```

12 | Complete the terminal row – Solution

```
for (int i = 0; i <= N; ++i)
{
    Price[i] = pOption->Payoff(
        Model.CalculateAssetPrice(N, i));
}
```

Boundary condition

At expiry there is no future continuation value. The option value equals its payoff at every terminal node.

GUIDED CODE PriceBySnell

Fill the three missing expressions

```
for (int n = N - 1; n >= 0; --n)
{
    for (int i = 0; i <= n; ++i)
    {
        double contVal =
            (q * /* up value */ + (1.0 - q) * /* down value */)
            / Model.GetR();

        double exerciseVal = pOption->Payoff(
            Model.CalculateAssetPrice(n, i));

        Price[i] = /* choose the larger value */;
    }
}
```

13 | Complete backward induction – Solution

```
for (int n = N - 1; n >= 0; --n)
{
    for (int i = 0; i <= n; ++i)
    {
        double contVal =
            (q * Price[i + 1] + (1.0 - q) * Price[i])
            / Model.GetR();

        double exerciseVal = pOption->Payoff(
            Model.CalculateAssetPrice(n, i));

        Price[i] = exerciseVal;
        if (contVal > Price[i]) Price[i] = contVal;
    }
}
```

14 | Record the stopping decision – Problem

GUIDED CODE Exercise policy

Define a precise flag

```
double contVal = /* discounted continuation */;
double exerciseVal = /* immediate payoff */;

priceTree[n][i] = /* node value */;
stoppingTree[n][i] = /* 1 or 0 */;
```

Convention

Store 1 only when immediate exercise is optimal and the exercise value is positive. Otherwise store 0.

14 | Record the stopping decision – Solution

```
priceTree[n][i] = exerciseVal;
stoppingTree[n][i] = (exerciseVal > 0.0);

if (contVal > priceTree[n][i])
{
    priceTree[n][i] = contVal;
    stoppingTree[n][i] = 0;
}
```

Tie convention

With the strict test `contVal > exerciseVal`, a positive tie is recorded as exercise. State the convention if ties matter in an application.

One vector or the full tree?

Structure	Storage	What it supports
`vector<double>`	$O(N)$	time-zero price only
triangular price tree	$O(N^2)$	inspect every node value
price + stopping trees	$O(N^2)$	inspect value and exercise policy

Choose for the question

Use the one-dimensional vector when only the root price is needed. Store the full trees when the exercise boundary or diagnostics matter.

STL containers: a practical map

Category	Containers in Topic 6	Main access idea
Sequence	`vector`, `list`	position/order
Adapters	`stack`, `queue`, `priority_queue`	restricted ends
Associative	`set`, `map`	key

Template idea

The same container design stores different element types: `vector<double>`, `vector<int>`, or `map<string, Option>`.

15 | Choose a container – Problem

GUIDED CODE STL survey

Match each task

Choose among `vector`, `list`, `stack`, `queue`, `set`, and `map`.

1. Store binomial node values and access them by integer index.
2. Process orders in first-in, first-out order.
3. Store unique ticker symbols.
4. Find an option record by contract identifier.
5. Undo the most recent action first.

15 | Choose a container – Solution

Task	Container	Reason
indexed node values	`vector`	direct index access
FIFO orders	`queue`	front out, rear in
unique tickers	`set`	unique keys
record by identifier	`map`	key-value lookup
undo history	`stack`	last-in, first-out

A `list` is useful for sequential traversal with efficient insertion or removal at a known position; it does not support direct indexing.

16 | Predict stack and queue output – Problem

GUIDED CODE Adapter containers

Trace the removal order

```
std::stack<int> s;
s.push(10); s.push(20); s.push(30);
std::cout << s.top();
s.pop();
std::cout << " " << s.top();

std::queue<int> q;
q.push(10); q.push(20); q.push(30);
std::cout << " " << q.front();
q.pop();
std::cout << " " << q.front();
```

16 | Predict stack and queue output – Solution

```
// Output:
30 20 10 20
```

Stack

`top()` observes the last item pushed.
LIFO: 30, then 20.

Queue

`front()` observes the oldest item.
FIFO: 10, then 20.

17 | Complete the iterator loop – Problem

GUIDED CODE Traversal — Fill dereference and increment

```
std::vector<int> values = {4, 7, 9};

for (auto it = values.begin();
     it != values.end();
     /* move to next item */)
{
    std::cout << /* current item */ << " ";
}
```

Question

What does `end()` represent? Is it itself a valid element?

17 | Complete the iterator loop – Solution

```cpp
for (auto it = values.begin();
     it != values.end();
     ++it)
{
    std::cout << *it << " ";
}
// Output: 4 7 9
```

Iterator vocabulary

`begin()` refers to the first element. `end()` refers one position past the last element and must not be dereferenced.

18 | What does `map::operator[]` do? – Problem

GUIDED CODE Key-value container — Predict size and value

```
std::map<std::string, int> counts;
counts["call"] = 3;

std::cout << counts.size() << std::endl;
std::cout << counts["put"] << std::endl;
std::cout << counts.size() << std::endl;
```

Question

Does reading the missing key `"put"` leave the map unchanged?

18 | What does `map::operator[]` do? – Solution

```
// Output:
1
0
2
```

Insertion on a missing key

`counts["put"]` creates the missing entry with a value-initialized `int`, which is zero.

Use `find()` to test for a key without inserting it. Use `at()` for checked access to an existing key.

Exit ticket: connect finance, algorithm, and storage

```
double contVal = /* discounted next-row values */;
double exerciseVal = pOption->Payoff(
    Model.CalculateAssetPrice(n, i));

priceTree[n][i] = /* ? */;
stoppingTree[n][i] = /* ? */;
```

Explain before leaving

What financial decision is encoded? Why is row `n` sized `n + 1`? When would one `vector<double>` be preferable to the full trees?

Reserve workshop

Build the full American put price and stopping trees.

R1 | Design the diagnostic overload – Challenge

GUIDED CODE Previous-year workshop — Return a price and fill two trees

```
double PriceBySnell(
    const BinomialTreeModel& Model,
    /* output price tree */,
    /* output stopping tree */);
```

Task

Choose parameter types that let the function resize and fill two vector-of-vector objects owned by the caller.

R1 | Design the diagnostic overload – Solution

```
double PriceBySnell(
    const BinomialTreeModel& Model,
    std::vector< std::vector<double> >& priceTree,
    std::vector< std::vector<int> >& stoppingTree);
```

Why non-const references?

The function must modify the caller's two trees. The model remains a read-only `const` reference.

R2 | Allocate and initialize the trees – Challenge

GUIDED CODE Previous-year workshop

Complete both phases

```
priceTree.resize(/* ? */);
stoppingTree.resize(/* ? */);
for (int n = 0; n <= N; ++n)
{
    priceTree[n].resize(/* ? */);
    stoppingTree[n].resize(/* ? */);
}

for (int i = 0; i <= N; ++i)
{
    priceTree[N][i] = /* terminal payoff */;
    stoppingTree[N][i] = /* positive payoff flag */;
}
```

R2 | Allocate and initialize the trees – Solution

```
priceTree.resize(N + 1);
stoppingTree.resize(N + 1);
for (int n = 0; n <= N; ++n)
{
    priceTree[n].resize(n + 1);
    stoppingTree[n].resize(n + 1);
}

for (int i = 0; i <= N; ++i)
{
    priceTree[N][i] = pOption->Payoff(
        Model.CalculateAssetPrice(N, i));
    stoppingTree[N][i] = (priceTree[N][i] > 0.0);
}
```

Here 1 means a positive exercise payoff. If a report marks every expiry node as stopping, document that different terminal convention.

R3 | Fill value and stopping trees – Challenge

GUIDED CODE Previous-year workshop

Complete one backward node

```
double contVal = /* use row n+1 */;
double exerciseVal = /* payoff at node (n,i) */;

priceTree[n][i] = exerciseVal;
stoppingTree[n][i] = /* positive exercise */;

if (/* continuation is better */)
{
    priceTree[n][i] = /* ? */;
    stoppingTree[n][i] = /* ? */;
}
```

R3 | Fill value and stopping trees – Solution

```
double contVal =
    (q * priceTree[n + 1][i + 1]
     + (1.0 - q) * priceTree[n + 1][i])
    / Model.GetR();
double exerciseVal = pOption->Payoff(
    Model.CalculateAssetPrice(n, i));

priceTree[n][i] = exerciseVal;
stoppingTree[n][i] = (exerciseVal > 0.0);

if (contVal > priceTree[n][i])
{
    priceTree[n][i] = contVal;
    stoppingTree[n][i] = 0;
}
```

R4 | Count exercise nodes by step – Challenge

GUIDED CODE Previous-year workshop

Reduce each row to one count

```
std::vector<int> exerciseCount(N, 0);

for (int n = 0; n < N; ++n)
{
    for (int i = 0; i <= n; ++i)
    {
        /* add stoppingTree[n][i] */
    }
}
```

Question

Why does `exerciseCount` have size `N` rather than `N + 1`?

R4 | Count exercise nodes by step – Solution

```
for (int n = 0; n < N; ++n)
{
    for (int i = 0; i <= n; ++i)
        exerciseCount[n] += stoppingTree[n][i];
}
```

Lecture parameters, American put

Root price ≈ 11.724. Exercise counts for steps 0 through 7 are

$$0,\ 0,\ 0,\ 0,\ 1,\ 2,\ 2,\ 3.$$

Expiry is excluded because the exercise decision there is forced rather than early.

Recitation 8 cheat sheet

Concept	Working rule
American node value	maximum of continuation and immediate payoff
`vector<T>(n)`	create `n` value-initialized elements
`push_back(x)`	append one new element and increase the size
Valid vector index	from `0` through `size()-1`
Triangular tree row	row `n` has `n+1` nodes
Iterator range	`begin()` is first; `end()` is one-past-last
`map[key]`	access or insert a value for `key`
Full stopping tree	$O(N^2)$ storage for the exercise policy